Simplifying the K215 correctness proofs

A source pinned research proposal for ProveIt

23 October 2026

Source baseline. `cbf94cb552a3b49509e0eee80e7455d4eb5974c6`.

Scope. This report concerns shortening and simplifying the existing correctness proofs. It does not propose a smaller Turing machine.

Validation boundary. Proposed Lean and Rocq proof changes require compilation in the repository's pinned toolchains. Static inspections and finite checks are supplementary evidence, not replacement kernel proofs.

Abstract

The most useful next simplifications are to remove legacy quotient adapters from Lean's primary semantic dependency path, and to share the branch-independent prefix of each decrement certificate before its zero/nonzero oracle split. Both retain the exact K215 machine and existing endpoint statements. At the pinned sources, an uncompiled 12-file Lean patch reduces the selected endpoint closure from 174 to 168 modules, removing 531 lines from that closure while retaining compatibility wrappers. A new finite diagnostic checks all 1,425 decrement pairs. Sharing reduces abstract replay calls from 1,337,729 to 852,647 in the Rocq full-site path and from 514,073 to 312,947 in the Lean full-start path. These are static and finite results, not new kernel checks or measured build-time improvements. The archive contains the proposed patch, candidate interfaces, pinned input data, new checkers, and an immutable integrity manifest.

Recommended order. First compile the dependency-only Lean patch and preserve existing clients. Then formalize the generic prefix/fork composition lemma once and replace duplicated decrement replay. Keep any larger calibration rewrite or shared C222 algebra project separate until these smaller changes are measured.

Proposal	Established in this package	Not established
Generic semantic spine	174 to 168 selected modules; 27,355 fewer source bytes and 531 fewer lines in endpoint import closure	Lean elaboration; preserved elaboration performance; smaller total maintained source
Decrement prefix sharing	1,425 pairs replayed; 36.26% fewer Rocq full-site calls and 39.12% fewer Lean full-start calls	Rocq or Lean kernel proof; build-time speedup; fewer source obligations

A necessary accounting distinction. The semantic-spine patch preserves legacy adapters for old clients. The overall patch adds 1,152 bytes and 46 lines across the affected source files. Its benefit is a smaller, cleaner primary dependency path, not a claim of deleted mathematical content.

Contents

1 The present proof and the scope of this proposal **3**

1.1 A current source pin rather than a historical receipt 3

1.2 Changes already integrated 3

2 Separate the generic semantic spine from quotient adapters **4**

2.1 Why the existing split is incomplete 4

2.2 The mathematical boundary already present in the proofs 5

2.3 What the dependency cut changes 5

2.4 Why compilation is still essential 6

3 Share the decrement prefix before the oracle split **6**

3.1 The remaining duplication 6

3.2 The exact soundness interface 6

3.3 The decrement fork 7

3.4 Separate measurements for the two production paths 8

4 Validation and integration plan **8**

4.1 Keep four kinds of evidence separate 8

4.2 A low-risk sequence of changes 9

4.3 Run the portable checks without changing the archive 10

5 Further research after these two changes **10**

6 Source pointers and artifact map **11**

1 The present proof and the scope of this proposal

The accepted K215 candidate has 215 working states, two tape symbols, a single two-way infinite tape, blank input, initial state zero, and a separate halt. Initialization is included in the count. The two formal lanes are intended to certify the same literal dense transition table, not merely machines of equal size.

The existing endpoint is

$$\mathrm{Halt}(K215, \mathrm{blank}) \quad \Longleftrightarrow \quad \exists d \ \mathrm{Derivation}_{\mathsf{ZFC}}(d, \bot).$$

Negating both sides gives nonhalting if and only if syntactic consistency, $\mathrm{Con}(\mathsf{ZFC})$. Consistency is not a premise of the equivalence. The repository explicitly describes these as external proof-assistant theorems; it does not supply an internal PA proof or a derivation encoded in ZFC's own proof syntax. Provability *in ZFC* therefore remains a separately stated formalization obligation. None of the refactorings below discharges it.

The endpoint composes five interfaces: dense-state renaming, blank initialization, binary/source simulation, source/program correspondence, and proof enumeration with ZFC calibration. This decomposition should be preserved. The proposals change how those interfaces are proved or imported, not their statements, the machine table, the source program, or the calibration target.

1.1 A current source pin rather than a historical receipt

All new source-based observations are pinned to `cbf94cb552a3b49509e0eee80e7455d4eb5974c6`. The review checked Git blob identities and SHA-256 digests for 203 current Lean source snapshots. The transitive project-source import closure of `KEndpoint215` is 174 modules, 1,752,650 bytes and 37,648 lines. These are static source measurements, excluding external toolchain libraries.

The earlier comparison pin is `6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95`. Its corresponding closure was 178 modules, 2,093,112 bytes and 40,517 lines. Thus 340,462 bytes and 2,869 lines of selected closure have already disappeared before the new proposals in this report. The historical phase-15 count of 176 modules is not the present count.

The repository records full Lean and Rocq builds at an earlier commit, `613f9cf28a`, followed by narrower checked updates and reuse records. Those records are valuable provenance, but their existence does not establish a full build of all source bytes at the present pin. We did not run Lean or Rocq. New patches below are explicitly *uncompiled candidates*.

1.2 Changes already integrated

Earlier proposals are not new recommendations here. The current source history already includes the unified QHyp quotient, computed proof trees, direct C222 compiler steps in both proof assistants, shared Rocq implication evaluation, normalized one-sided Rocq certificates, AX successor inference, and a direct full-seed construction. A successful next refactoring should start from these current implementations rather than reproduce an obsolete plan.

Integrated change	Introducing commit
Unified QHyp quotient	`8e46744`
Computed proof trees	`775267c`
Direct compiler in Lean	`fa92b88`
Direct compiler in Rocq	`7b4ee981`
Shared Rocq implication evaluation	`85cbbc9`
One-sided normalized Rocq certificates	`16a674f`
AX successor inference	`4fb82d6`
Direct full-seed construction	`ff928ef0`

The packaged source ledger and integration map give full hashes and paths. The old phase-15 checkpoint and previous proposal documents are historical evidence, not validation of this report's current source pin.

2 Separate the generic semantic spine from quotient adapters

2.1 Why the existing split is incomplete

The current proof has already extracted a `WFModelCore`, but its imported mathematical lemmas still bring the legacy CP/GIC quotient implementation into the primary endpoint closure. Two concrete dependencies, with arrows denoting transitive import reachability, are

$$\texttt{WFModelCore} \longrightarrow \texttt{WellFoundedPart} \longrightarrow \texttt{QuotientCPGIC},$$
$$\texttt{FiniteSets} \longrightarrow \texttt{SingletonInfinity} \longrightarrow \texttt{QuotientBasic}.$$

The general mathematics and its particular quotient instantiation share files. Merely introducing a top-level core therefore does not create a dependency boundary.

The candidate separates four modules at their first quotient-specific adapter. The generic declaration bodies move verbatim, preserving their namespaces; the old modules become wrappers that import the core and retain the quotient-specific tail. Existing import paths and declaration names therefore remain available to compatibility clients. This avoids rewriting their mathematical proofs while allowing the primary path to use only the generic layer.

Existing module	New core module	First retained adapter boundary
`SingletonInfinity`	`SingletonInfinityCore`	`theorem quotientBase`
`FiniteSets`	`FiniteSetsCore`	`theorem quotientBase`
`InductiveAttempts`	`InductiveAttemptsCore`	`theorem quotientFiniteBase`
`WellFoundedPart`	`WellFoundedPartCore`	Actual-quotient GIC and base adapter

Four further edits redirect `WFRelativization` and `KQ1ZF` to the new cores, and replace `KLowerQHyp`'s use of `QuotientModel.memberForm` and its satisfaction lemma with the already existing `KQ1Model` versions. The compatibility module `WFModel` also gains an explicit `WellFoundedPart` import for its quotient adapter, replacing a formerly transitive dependency. The syntax change in `KLowerQHyp` is a specific interface substitution, not a claim that arbitrary quotient lemmas are interchangeable. The patch and checks record the exact occurrences.

2.2 The mathematical boundary already present in the proofs

The generic layer has a concrete semantic interface, not an empty abstraction. Let V be a carrier with membership relation $\in_M$.

1. `SingletonInfinity.Base` requires extensionality, exact pairs and formula-indexed Separation. Its least singleton-closed index construction additionally uses a root and an already supplied singleton-closed container. It does not depend on a quotient representation.
2. `FiniteSets.Base` adds exact Union. The finite-set constructors and represented graph formulas use these fields rather than the particular CP/GIC construction.
3. `ordinary_infinity_of_container` takes that finite base, formula-indexed functional Replacement, an empty root e, and a set Y containing e and closed under exact singleton. It constructs an ordinary von Neumann-successor inductive set. The existing theorem has no quotient or CP/GIC argument.
4. `wf_closure` takes the finite base, the single first-order instance

$$\texttt{Sat}(\in_M, \lambda_.a, \texttt{gicForm}(\texttt{witnessForm})),$$

   and the hypothesis that every member of a lies in the internally well-founded part. The witness formula describes a transitive internally well-founded witness containing its input. Collection gathers the witnesses; their union is internally well-founded; adjoining a supplies its witness. This uses represented subsets and the stated formula instance, not external well-foundedness or Collection for arbitrary external predicates.

The adapter layer then instantiates precisely these hypotheses using the CP/GIC quotient. The dependency direction should follow that reasoning: the representation-specific instance imports the general theorem, not conversely. The candidate moves the existing proof bodies unchanged across this boundary. It does not add CP as a hidden hypothesis, claim CP was previously assumed by the generic theorem, or replace a proof with a stronger semantic assumption.

2.3 What the dependency cut changes

At the pinned sources, the candidate removes ten old module names from the selected endpoint closure and introduces the four cores. The six-module net reduction is 174 to 168. The old names absent from this closure are `FiniteSets`, `InductiveAttempts`, `SingletonInfinity`, `WellFoundedPart`, and the six `QuotientBasic`, `QuotientCPGIC`, `QuotientModel`, `QuotientPower`, `QuotientReplacement`, `QuotientSeparation` modules.

Selected source decreases from 1,752,650 to 1,725,295 bytes and from 37,648 to 37,117 lines. This is a structural dependency result. The patch neither deletes those six mathematical theories from the repository nor asserts that Lean automatically discards every unused declaration. Legacy clients can continue to import their wrappers.

Cost. The overall patch adds 1,152 bytes and 46 lines across its affected files because wrappers and new imports are added. This small duplication buys a clear API boundary and avoids a breaking repository-wide migration. If every legacy client is later migrated, wrapper retirement becomes a separate decision with its own evidence.

2.4 Why compilation is still essential

Verbatim movement of declarations is strong static evidence but is not a Lean elaboration theorem. Import changes may alter transitive availability, typeclass instances, notation, reducibility, or namespace behavior. The patch has not been compiled here. The static audit establishes source identity, declaration-body preservation, import resolution and the stated census; it cannot certify elaborated proof terms.

Adopt the dependency patch first. Compile each core, each compatibility wrapper, both endpoint clients, and the retained compatibility scope in the pinned Lean toolchain. Compare `#print axioms` output with the accepted baseline and preserve the exact endpoint statements. Only then measure clean and incremental build time, peak memory and proof-object size. A reduced import census alone warrants none of those performance claims.

3 Share the decrement prefix before the oracle split

3.1 The remaining duplication

A decrement instruction has two outcomes: zero and nonzero. Current symbolic certificates replay each outcome from the same site-entry configuration, although decoding and register-selection work before the branch oracle is identical. This duplication remains after the already integrated one-sided normalization and shared implication changes. The proposal establishes the common premise once and forks only at the branch-sensitive operation; it does not merge the two outcome obligations or guess the branch.

The selected construction uses the existing sound symbolic executor in *neutral* mode for a prefix, then the same executor in a branch-specialized mode for each continuation. There is no new tape semantics and no new branch-noninterference induction. The only new semantic argument composes two existing `exec_sound` applications.

3.2 The exact soundness interface

Write $[\![s]\!]_{n,v}$ for the existing concretization of a symbolic configuration s, at natural parameter n and register valuation v. Let $\rightsquigarrow^*$ and $\rightsquigarrow^+$ denote concrete reflexive and positive reachability. The existing executor theorem has the following shape:

$$\begin{aligned}&\mathtt{Valid}(b,v)\ \wedge\ \mathtt{exec}(b,T,f,s,m)=\mathtt{some}(t)\\ &\quad\Longrightarrow\quad T(t)=\mathtt{true}\ \wedge\ [\![s]\!]_{n,v}\rightsquigarrow^*[\![t]\!]_{n,v}\ \wedge\ (m=\mathtt{false}\Rightarrow[\![s]\!]_{n,v}\rightsquigarrow^+[\![t]\!]_{n,v}).\end{aligned}$$

Here b is either neutral or a specified register/outcome branch, T is the target predicate, f is fuel, and m records whether positive progress is already established. Ordinary register validity for a branch implies neutral validity, since the latter imposes no branch predicate.

Proposition 3.1 (Sound cut composition)**.** *Assume the existing box soundness hypotheses and*

$$\begin{aligned}\mathtt{exec}(\mathtt{none},C,f,s,\mathtt{false})&=\mathtt{some}(c),\\ \mathtt{exec}(b,T,g,c,\mathtt{true})&=\mathtt{some}(t),\\ \mathtt{Valid}(b,v).&\end{aligned}$$

Then $T(t)=\mathtt{true}$ *and* $[\![s]\!]_{n,v}\rightsquigarrow^+[\![t]\!]_{n,v}$*. The cut predicate* C *may be arbitrary.*

Proof. Derive neutral validity from the register-validity component of $\texttt{Valid}(b, v)$. Apply existing executor soundness to the neutral prefix. Because its initial progress flag is false, this gives positive reachability from $[\![s]\!]$ to $[\![c]\!]$. Apply executor soundness a second time, with the original branch validity, to the continuation. It gives $T(t)$ and reflexive reachability from $[\![c]\!]$ to $[\![t]\!]$. Compose positive and reflexive reachability. A reflexive continuation is permitted; positive execution is already supplied by the prefix. □

We start the continuation with `moved=true`. Requiring each suffix to establish new positive progress would be an unnecessary strengthening of the original contract and could reject a valid cut whose continuation is reflexive. Starting the *prefix* with true would instead lose the positive-progress conclusion needed by the site contract.

3.3 The decrement fork

Use the cut predicate

$$C_r(q, \ell, L, R) \;\equiv\; q = 0 \;\wedge\; \ell = \texttt{false} \;\wedge\; R = [\texttt{box}(\texttt{mark}(r))].$$

Test this explicit predicate before taking a step. A failed neutral oracle is not a sound generic stopping rule: the existing stepper may fall back to a table transition. The boundary controls include a synthetic table exhibiting this fallback. The left segment list L is retained, not erased or normalized without proof. The register r is the physical register after the existing register permutation.

The proposed Boolean `forkTo` / `fork_to` first runs the neutral prefix to C_r with `moved=false`. Failure returns false. Given the resulting c, it requires both specialized continuations from c, with `moved=true`, to match the original zero and nonzero target configurations. To prove either old site contract, take the corresponding conjunct and apply sound cut composition. Existing `sMatches_conc` / `matches_conc` translates accepted symbolic matching into equality of concrete endpoints.

This proof does not require proving that neutral execution syntactically agrees with both original full traces. Neutral soundness already proves the concrete prefix for every valid valuation. The independent finite diagnostic checks trace agreement to establish the proposed sharing opportunity and quantify work; that comparison is not an extra formal premise.

The archive includes an uncompiled Lean candidate module, an exact decrement-branch fragment, and an uncompiled Rocq integration fragment. The Lean cut module imports only `KHomeReady`, takes fuel explicitly, and sits below `KSim` in the dependency graph, so `KSim` can import it without a cycle. The Rocq fragment reuses the existing four box hypotheses and is intended to be inserted around the existing `Interpreter230Sites.v` soundness section, not compiled as a standalone file. The public Rocq `check_site`/`check_edge` and Lean `checkEdge` signatures, together with site-soundness statements, can stay unchanged, but the Boolean checker implementation changes. The literal `all_leaves_check` certificate and all downstream endpoints must be regenerated and rebuilt. Keeping a theorem's statement is not a license to reuse a certificate whose checked expression changed. Two separately fuel-bounded phases may accept different inputs from the old single phase at fixed fuel; only the universal semantic contract is intended to remain unchanged.

3.4 Separate measurements for the two production paths

The current Rocq production path executes a full site without the Lean Home shortcut. Lean's current full-start path uses that shortcut. Their abstract replay counts must therefore be reported separately.

Quantity	Rocq full site	Lean full start with Home
Decrement sites	1,425	1,425
Complete branch outcomes	2,850	2,850
Original abstract replay calls	1,337,729	514,073
Calls after prefix sharing	852,647	312,947
Duplicate calls removed	485,082	201,126
Fraction removed	36.26%	39.12%

These are finite diagnostic counts, not formal compilation measurements. The newly authored Python checker parses pinned source/data and transcribes the abstract transition rules; it does not execute repository programs. It equates the current Lean and Rocq tables, checks all 430 enabled actions against the native table under the explicit state-label map, equates the two 179-node source programs, regenerates all 4,751 leaves, and replays both complete decrement outcomes, locates their common prefix at the expected physical-register cut, checks neutral prefix execution, and checks positive prefix progress. Fuel exhaustion, failure to reach the expected target, and unexpected oracle behavior are failures, not successes.

The transcription is a distinct trust boundary: agreement of pinned data and finite traces is evidence about these cases, not a kernel proof of the Lean or Rocq executor. The counts omit target-predicate evaluation and normalization work and add 1,425 successful prefix-cut tests. Eleven rejection/progress controls exercise wrong cuts, wrong branches, empty-prefix progress and fallback behavior. Abstract calls are not concrete machine steps, proof-term size, kernel reductions, wall-clock time or peak memory. All 4,751 leaves and 6,176 original guarded outcomes remain; only the 2,850 decrement outcomes are measured here. The reported percentage covers decrement work only, not every source-site kind or the full endpoint.

4 Validation and integration plan

4.1 Keep four kinds of evidence separate

1. **Current source identity.** The archive's source manifests bind copied input bytes to the exact Git commit, Git blob IDs, SHA-256 digests, paths and verified source URLs. A source hash establishes which text was inspected, not its correctness.
2. **Static refactoring evidence.** The source-census and generic-spine checks establish import closure, preserved source bodies, selected substitutions and the stated size accounting. They do not elaborate declarations or validate typeclass behavior.
3. **Finite diagnostic evidence.** The newly authored replay checker establishes its reported finite cases under its transcribed abstract semantics. It is neither a proof-assistant certificate nor an exhaustive check of all natural register values by concrete simulation.

4. **Formal acceptance.** The maintained Lean and Rocq checkers must accept the changed source in their pinned toolchains, with source-bound records of endpoint statements and assumptions. This last step has not been performed for the proposals.

Record	What it supports	What it does not support
Historical full builds	Repository-reported Lean 4.32.0 and Rocq 9.1.0 checks of their bound historical bytes	All later source edits automatically passing
Later integration records	Selective serial Lean builds; normalized Rocq adoption reports the 215 compile order; later slices report narrower checks	A newly repeated all-current Lean build or current recursive `coqchk`
This research package	Pinned source integrity, static census, patch audit and finite replay measurements	New Lean/Rocq kernel checks or timing results

The normalized Rocq adoption explicitly reports that recursive `coqchk` was not rerun. Later endpoint rechecks against an already compiled reference are not equivalent to a fresh recursive audit of every present input. The current repository history, rather than older proposal-catalogue status text, determines which simplifications are already integrated.

4.2 A low-risk sequence of changes

1. **Establish a current baseline.** Record exact source bytes, compiler versions, complete dependency selection, endpoint statements and assumption reports. Perform a full current build before attributing a regression or speedup to these proposals.
2. **Land only the semantic-spine patch.** Apply the 12-file candidate to the exact pin in a fresh branch. Compile the new cores, four legacy wrappers, the explicit `WFModel` compatibility import, and both endpoint clients. Include the separate retained compatibility scope, because it is intentionally not covered by the primary import census.
3. **Add fork composition without replacing production checks.** Compile the new composition lemmas against unchanged executor soundness. Check zero, nonzero, neutral validity, the positive-progress flag, and normalized endpoint equality independently of performance.
4. **Replace only decrement replay.** Preserve the source-site coverage theorem, register/path guards, physical-register map and branch contracts. The shared Lean `KSim` checker also serves non-215 K-family clients, so rebuild their affected site certificates and clients as a separate runtime-compatibility gate; no finite success is claimed here for those other variants. Regenerate changed literal certificates and rebuild all dependent endpoints. For Rocq, repeat the recursive kernel audit and endpoint assumption reports; for Lean, record endpoint `#print axioms` and recheck typed endpoint clients.
5. **Measure before expanding.** Compare clean and incremental builds under the same toolchain, cache policy, CPU allocation and memory limit. Report total maintained source, selected closure, check time, object size and peak memory separately. Keep a rollback path for each change.

4.3 Run the portable checks without changing the archive

Extract a fresh copy and run `python3 verify.py`. The fixed `MANIFEST.sha256` lists every payload file other than itself. The verifier checks exact membership and hashes, refuses optimized Python, and runs only the selected newly authored checkers in a temporary copy. It compares the original package before and after, including manifest bytes, and checks that regenerated diagnostic results match the archived bytes. No Lean/Rocq source, repository generator, downloaded script, or build receipt reader is executed.

The manifest is an integrity record, not a cryptographic signature. The verifier is a research harness, not a sandbox. It does not regenerate or repair its own baseline manifest. It rejects extra files, so edits and document rebuilding should be performed in a separate working copy. The historical baseline is retained as a compact import/hash/metric ledger rather than a second full source tree; the verifier checks that ledger's graph and sums, not absent historical bytes.

To rebuild the PDF from editable sources, run `make pdf` with a standard TeX installation. Shell escape is disabled. A rebuilt PDF may differ byte-for-byte across TeX distributions; that is separate from whether the mathematical text and checks reproduce. The shipped PDF is rendered and visually inspected; no TeX compiler log or page-image collection is needed in the repository archive.

5 Further research after these two changes

1. **How much kernel work is actually saved by the fork?** The finite replay counts predict less duplicated computation, but evaluator sharing, proof-term representation and normalization cost may dominate. Measure whether the shared prefix survives elaboration and kernel checking rather than being expanded twice by generated terms.
2. **Can prefixes be shared across distinct sites?** This report shares only the two outcomes of each decrement. A trie or DAG of common decoder prefixes might share work between sites, but must preserve exact source PCs, left frames, physical-register identities, fuel behavior and positive progress. Quantify the extra checker and soundness-lemma cost before proposing a new certificate format.
3. **When should compatibility adapters be retired?** The dependency cut is deliberately additive in total source. A later repository-wide client inventory could determine whether wrappers remain useful. Removing them would require a separate migration and compatibility test, not merely the smaller endpoint census.
4. **Is a shared C222 compiler algebra worthwhile now?** Direct C222 compilation is already integrated. Its remaining Lean proofs are only about 30 and 37 lines. A generic many-field interface and two adapters may cost more than the duplication it removes. Defer this abstraction until total helper, wrapper and dependency costs show a clear benefit.
5. **What exactly is needed for an internal ZFC equivalence?** The existing external proof-assistant theorem and the intended theorem encoded inside ZFC are distinct deliverables. Specify the internal machine semantics, proof coding and derivation target, then audit which current lemmas admit the required internal formalization. This is not a claim that ZFC proves its own consistency or the machine's nonhalting.

The first acceptance milestone is modest and concrete: an unchanged K215 endpoint statement, a passing retained compatibility scope, and fresh source-bound kernel checks for the dependency cut

and decrement fork. More ambitious abstractions should be justified by measurements after that milestone, not by repeating proposals the repository has already adopted.

6 Source pointers and artifact map

The source pin on the first page applies throughout. These links point to immutable source views, not the moving main branch. Full paths, Git blob identities and SHA-256 values are recorded in the packaged manifests.

- K215 verification overview: machine identity, external theorem statement and historical full-build scope.
- `SingletonInfinity.lean`, 136–154: Generic/quotient split boundary.
- `FiniteSets.lean`, 157–163: Generic/quotient split boundary.
- `InductiveAttempts.lean`, 321–347: Generic/quotient split boundary.
- `WellFoundedPart.lean`, 219–235: Quotient closure and nonempty-domain adapters.
- `KLowerQHyp.lean`, 140–148: Member-form facade use replaced by the patch.
- `WFModel.lean`, 1–21: Compatibility theorem requiring explicit adapter import.
- `KRuntimeCore.lean`, 432–459: Executor soundness and positive-progress premise.
- `KHomeReady.lean`, 92–100: Lean Home-ready executor soundness.
- `Runtime230Core.v`, 382–418: Rocq executor soundness.
- `Interpreter230Sites.v`, 45–50: Current Rocq decrement checker.
- `Interpreter230Sites.v`, 130–147: Current Rocq decrement contract.
- `Interpreter230Check.v`, 16–31: Edge binding and the source-leaf certificate.

Artifact map. The report sources and PDF are at the archive root. `review/` contains the pinned-source and integration review, current-source snapshots, historical ledger and read-only verifier. `lean/` contains the generic-spine patch, candidate files, source-movement audit and reproducible builder. `rocq/` contains the new finite diagnostic, pinned runtime inputs and uncompiled fork candidates for both proof assistants. `audit/` contains separately written static and finite replays and their source reviews. The root `verify.py` and `MANIFEST.sha256` bind and check the complete package.

Original repository source notices are retained. The package does not contain old proposal archives, large build logs, external delivery records, compiled theorem objects, private notes or page-render images.